

Reporte Tecnico

Para el reporte técnico, en buena parte use ayuda de la IA, en si ya mencionado modo Pro en la versión Plus de Gemini, principalmente ciertos parámetros que no se vieron en clase o se vieron muy rápido como los arboles, ademas de que para mostrarlo en un sentido profesional, Gemini muestra los cálculos con la notación que no existe en Word mas que por extensiones, para facilitar la lectura se tomo esta decisión, apoyarnos de la IA para esta parte.

Por ello generalizaremos, el tipo de algoritmo que es, cuales de los algoritmos que hicimos entran en ese caso y como se procedera.

Divide y Vencerás.

Teorema Maestro

Algoritmo: Búsqueda Binaria (Paradigma: Divide y Vencerás)

Este algoritmo se utiliza en el planificador del sistema operativo para localizar rápidamente el índice de un proceso específico (PID) dentro del arreglo de procesos, asumiendo que este se encuentra ordenado.

Código Esencial:

```
C
int binary_search_process(Process arr[], int low, int high, int target_pid) {
    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid].pid == target_pid) return mid;
        if (arr[mid].pid < target_pid) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}
```

Explicación Narrativa: El algoritmo calcula el punto medio (mid) del espacio de búsqueda. Si el proceso en esa posición es el objetivo, finaliza. Si el PID buscado es menor, descarta toda la mitad derecha ajustando el límite superior ($high = mid - 1$). Si es mayor, descarta la mitad izquierda ($low = mid + 1$).

Al descartar la mitad de los elementos en cada iteración, el espacio de búsqueda se reduce de manera exponencial, mientras que las operaciones internas (comparaciones, sumas y divisiones) toman un tiempo constante.

Demostración Matemática (Teorema Maestro): La naturaleza recursiva/iterativa de este algoritmo se puede modelar con la siguiente ecuación de recurrencia:

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Donde:

- n es el tamaño del arreglo de procesos.
- $a = 1$: En cada paso, el algoritmo solo resuelve **un** subproblema (busca en la mitad izquierda o en la mitad derecha, nunca en ambas).
- $b = 2$: El tamaño del problema original se divide a la **mitad** en cada paso.
- $f(n) = O(1)$: El costo de calcular **mid** y realizar las comparaciones en cada paso es constante. Esto implica que $d = 0$ (ya que $O(n^0) = O(1)$).

Evaluando la condición del Teorema Maestro:

$$a = b^d$$

$$1 = 2^0 \implies 1 = 1$$

Dado que $a = b^d$, nos encontramos en el **Caso 2** del Teorema Maestro. La fórmula para este caso es:

$$T(n) = \Theta(n^d \log n)$$

Sustituyendo $d = 0$:

$$T(n) = \Theta(n^0 \log n) = \mathbf{O(\log n)}$$

Algoritmo: Compactación de Memoria (Paradigma Lógico: Divide y Vencerás)

Utilizado por el Administrador de Memoria para resolver la fragmentación externa. Aunque conceptualmente divide la memoria en bloques "ocupados" y "libres" para reorganizarlos, su implementación práctica es secuencial sobre una Lista Doblemente Ligada.

Código Esencial:

C

```
void divide_conquer_compact(MemoryManager* mm) {
    MemoryBlock* current = mm->head;
    int total_occupied = 0;

    // Fase 1: Calcular espacio ocupado
    while (current != NULL) {
        if (current->free == 0) total_occupied += current->size;
        current = current->next;
    }

    // Fase 2: Liberar nodos fragmentados
    current = mm->head->next;
    while (current != NULL) {
        MemoryBlock* temp = current;
        current = current->next;
        free(temp);
    }

    // Fase 3: Reconstruir cabecera (Omisión de código de asignación por brevedad)
    // ... Creación de bloque consolidado ...
}
```

Explicación Narrativa: El algoritmo opera en tres fases estrictamente secuenciales. En la Fase 1, recorre toda la lista para sumar la memoria ocupada. En la Fase 2, vuelve a recorrer la lista desde el segundo nodo para liberar la memoria fragmentada dinámicamente. En la Fase 3, reconstruye los punteros asignando un gran bloque ocupado y un gran bloque libre.

Demostración Matemática (Análisis por Sumatorias): Dado que no existe recursión ni división fraccionada del problema, analizamos las operaciones fundamentales mediante sumatorias para un conjunto de n bloques de memoria:

- **Costo de la Fase 1:** Un ciclo `while` que evalúa los n elementos. Las sumas y comparaciones toman un tiempo constante c_1 .

$$T_1(n) = \sum_{i=1}^n c_1 = c_1 \cdot n$$

- **Costo de la Fase 2:** Un ciclo `while` que recorre y libera $(n - 1)$ elementos. La operación `free()` toma un tiempo constante c_2 .

$$T_2(n) = \sum_{i=1} c_2 = c_2 \cdot (n - 1) = c_2 \cdot n - c_2$$

- **Costo de la Fase 3:** Operaciones de asignación de punteros y un bloque `if/else`, cuyo costo total máximo es una constante c_3 .

$$T_3(n) = c_3$$

Sumando las complejidades parciales:

$$T(n) = T_1(n) + T_2(n) + T_3(n)$$

$$T(n) = (c_1 \cdot n) + (c_2 \cdot n - c_2) + c_3$$

$$T(n) = n(c_1 + c_2) + (c_3 - c_2)$$

Aplicando las reglas asintóticas para descartar constantes y términos de menor orden:

$$T(n) \approx c \cdot n \implies \mathbf{O(n)}$$

Fuerza Bruta y Backtracking.

Demostración por sumatorias.

Algoritmo: Fuerza Bruta (Worst Fit para Asignación de Memoria)

Este algoritmo pertenece a la familia de búsqueda exhaustiva. Su objetivo es recorrer el mapa de memoria completo para encontrar el bloque libre más grande disponible (Peor Ajuste), lo cual es útil en escenarios donde se busca dejar un residuo de memoria lo suficientemente grande para procesos futuros.

Código Esencial:

C

```
int brute_force_worst_fit(MemoryManager* mm, int size) {
    MemoryBlock* current = mm->head;
    MemoryBlock* worst_block = NULL;
    int max_size = -1;

    while (current != NULL) {
        if (current->free == 1 && current->size >= size) {
            if (current->size > max_size) {
                max_size = current->size;
                worst_block = current;
            }
        }
        current = current->next;
    }

    if (worst_block != NULL) {
        worst_block->free = 0;
        worst_block->pid = worst_block->start;
        return worst_block->start;
    }
    return -1;
}
```

Explicación Narrativa: A diferencia de los algoritmos de división, la Fuerza Bruta no omite ninguna posibilidad. El algoritmo inicializa un puntero al inicio de la Lista Doblemente Ligada de memoria. Mediante un ciclo while, evalúa estrictamente cada uno de los nodos (bloques de memoria). Para cada nodo, realiza comprobaciones de estado (si está libre y si tiene el tamaño suficiente) y actualiza el bloque máximo encontrado. Al ser una búsqueda exhaustiva, el ciclo no se detiene al encontrar el primer candidato, sino que procesa todos los elementos hasta llegar a NULL.

Demostración Matemática (Análisis por Sumatorias): El rendimiento de un algoritmo iterativo se determina sumando el costo de las operaciones elementales que ocurren dentro de sus ciclos.

Sea n el número de bloques en la memoria (nodos en la lista ligada).

- Las inicializaciones antes del ciclo toman un tiempo constante c_1 .
- El ciclo while se ejecuta exactamente n veces en el peor y en el mejor de los casos (búsqueda exhaustiva).
- Dentro del ciclo, las evaluaciones if, comparaciones y la asignación $\text{current} = \text{current} \rightarrow \text{next}$ toman un tiempo constante c_2 .
- Las operaciones finales de asignación de pid y retorno toman un costo constante c_3 .

La ecuación de tiempo total $T(n)$ se define mediante la sumatoria del ciclo más las constantes externas:

$$T(n) = c_1 + \sum_{i=1}^n c_2 + c_3$$

Resolviendo la sumatoria:

$$T(n) = c_1 + (c_2 \cdot n) + c_3$$

$$T(n) = c_2 \cdot n + (c_1 + c_3)$$

Para determinar la notación Big-O asintótica, descartamos las constantes aditivas $(c_1 + c_3)$ y el coeficiente multiplicativo c_2 :

$$T(n) = \mathbf{O(n)}$$

Algoritmo: Backtracking (Simulación de Asignación con Pila)

El Backtracking es un algoritmo de retroceso. En nuestro Sistema Operativo, se utiliza para registrar en un "historial" los intentos de asignación de RAM. Si un proceso no logra obtener memoria, el sistema da un paso atrás (deshace la acción) extrayéndolo de la pila.

Código Esencial:

C

```
int backtracking_allocation(MemoryManager* mm, Stack* history, int pid, int size) {  
    // 1. Guardamos el estado actual en la Pila (Avanzar)  
    stack_push(history, pid);  
  
    // 2. Intentamos realizar la asignación de memoria  
    int address = brute_force_worst_fit(mm, size);  
  
    // 3. BACKTRACKING: Si la asignación falló, retrocedemos  
    if (address == -1) {  
        stack_pop(history);  
        return -1;  
    }  
  
    return address;  
}
```

Explicación Narrativa: El algoritmo se compone de tres pasos secuenciales. Primero, ejecuta la operación `stack_push`, la cual inserta un elemento al final de un arreglo de forma directa. Segundo, invoca al algoritmo de asignación de memoria (`brute_force_worst_fit`). Finalmente, evalúa una condición; si se cumple la falla, ejecuta `stack_pop`, retirando el último elemento del arreglo mediante la reducción simple de un índice. No existen ciclos recursivos ni iteraciones propias dentro de la función de Backtracking, por lo que su costo depende completamente de la función invocada internamente.

Demostración Matemática: Desglosamos el tiempo de ejecución sumando el costo de las subrutinas individuales:

- Operación `stack_push`: Costo constante $c1=O(1)$.
- Función `brute_force_worst_fit`: Como se demostró previamente, toma un tiempo dependiente de n (cantidad de bloques de memoria). Su costo es $c2 \cdot n=O(n)$.
- Operación `stack_pop` (en caso de falla): Costo constante $c3=O(1)$.

La función de costo total $T(n)$ se expresa como:

$$T(n) = T(\text{push}) + T(\text{worst_fit}) + T(\text{pop})$$

$$T(n) = c_1 + (c_2 \cdot n) + c_3$$

Agrupando los términos constantes:

$$T(n) = c_2 \cdot n + C$$

Por las propiedades de la notación Big-O:

$$T(n) = \mathbf{O(n)}$$

Algoritmos Greedy (Avaros).

SJF.

Algoritmo: Greedy - First Fit (Asignación de Memoria)

El paradigma Greedy (Avaro) se caracteriza por tomar la decisión localmente óptima en el momento actual con la esperanza de encontrar un óptimo global. En el caso de First Fit, el algoritmo toma el primer bloque de memoria que cumpla con el requerimiento de tamaño, sin gastar recursos de procesamiento en evaluar si existe una mejor opción más adelante.

Código Esencial:

C

```
int mm_allocate_first_fit(MemoryManager* mm, int size) {
    MemoryBlock* current = mm->head;

    while (current != NULL) {
        if (current->free == 1 && current->size >= size) {
            // Fragmentación (División del bloque)
            if (current->size > size) {
                // ... Creación de bloque restante (Costo O(1)) ...
            }
            current->free = 0;
            return current->start; // Decisión Greedy: Retorna inmediatamente
        }
        current = current->next;
    }
    return -1;
}
```

Explicación Narrativa: El algoritmo inicializa un puntero en la cabecera de la memoria. A través de un ciclo while, evalúa secuencialmente los nodos. La característica "Greedy" radica en que, en el instante en que la condición (`current->free == 1 && current->size >= size`) se evalúa como verdadera, el algoritmo fragmenta el bloque (operación de punteros constante), cambia el estado y ejecuta un return, interrumpiendo la iteración de inmediato.

Demostración Matemática (Análisis del Peor Caso): Para evaluar el rendimiento de un algoritmo de búsqueda con salida temprana, el análisis asintótico exige estudiar el "Peor Caso" (Worst Case Scenario). El peor caso ocurre cuando el bloque adecuado se encuentra en la última posición de la lista, o cuando no existe ningún bloque adecuado.

Sea n la cantidad de bloques en la lista ligada:

- La asignación inicial toma un tiempo constante c_1 .
- El ciclo ejecuta su bloque de instrucciones (comparaciones lógicas y salto de puntero) n veces. Asignamos a estas instrucciones un tiempo constante c_2 .
- La operación de fragmentación y retorno toma un tiempo c_3 .

La sumatoria para el tiempo total $T(n)$ en el peor caso es:

$$T(n) = c_1 + \sum_{i=1}^n c_2 + c_3$$

$$T(n) = c_1 + (c_2 \cdot n) + c_3$$

$$T(n) = c_2 \cdot n + (c_1 + c_3)$$

Ignorando constantes por definición de límite asintótico superior:

$$T(n) = \mathbf{O(n)}$$

Algoritmo: Greedy - Best Fit (Asignación de Memoria)

A diferencia de First Fit, el algoritmo Best Fit utiliza una estrategia Greedy orientada a minimizar el desperdicio (fragmentación interna/externa). Para ello, busca iterativamente a lo largo de toda la memoria y selecciona el bloque que deje el menor residuo posible.

Código Esencial:

```
C
int mm_allocate_best_fit(MemoryManager* mm, int size) {
    MemoryBlock* current = mm->head;
    MemoryBlock* best_block = NULL;
    int min_size = 9999999;

    while (current != NULL) {
        if (current->free == 1 && current->size >= size) {
            if (current->size < min_size) {
                min_size = current->size;
                best_block = current;
            }
        }
        current = current->next;
    }

    if (best_block != NULL) {
        // ... Asignación y fragmentación (Costo O(1)) ...
        return best_block->start;
    }
    return -1;
}
```

Explicación Narrativa: El flujo algorítmico es casi idéntico al algoritmo de Fuerza Bruta (Worst Fit), pero con una heurística de optimización invertida. Recorre la totalidad de la lista evaluando cada nodo. Si el nodo es libre, tiene espacio suficiente y su tamaño es menor que el `min_size` actual, se convierte en el nuevo bloque óptimo. Al requerir la minimización del residuo, el ciclo nunca puede interrumpirse tempranamente.

Demostración Matemática: Sea n el número total de fragmentos de memoria. Dado que el algoritmo debe leer absolutamente todos los nodos sin excepción para asegurar matemáticamente que ha encontrado el mínimo global, la ejecución del ciclo `while` siempre será exactamente n .

$$T(n) = c_{\text{inicio}} + \sum_{i=1}^n c_{\text{comparacion}} + c_{\text{fin}}$$

$$T(n) = c_{\text{inicio}} + (c_{\text{comparacion}} \cdot n) + c_{\text{fin}}$$

$$T(n) \approx c \cdot n$$

Conclusión: La complejidad temporal es estrictamente $O(n)$ tanto en el mejor, promedio y peor caso, ya que requiere recorrer toda la estructura de memoria. La complejidad espacial permanece constante en $O(1)$.

Algoritmo: Greedy - Shortest Job First (Planificador SJF)

El algoritmo SJF (Trabajo más corto primero) es la implementación Greedy clásica en la planificación de CPU. En cada ciclo de decisión, el planificador evalúa a todos los procesos disponibles en estado de espera y selecciona aquel que requiera el menor tiempo de ráfaga (burst time), optimizando así el tiempo de respuesta promedio del sistema.

Código Esencial:

```
C
int scheduler_next(Scheduler *scheduler) {
    // Si es tipo SJF (tipo == 2)
    if (scheduler->sjf_count == 0) return -1;

    int min_index = 0;
    for (int i = 1; i < scheduler->sjf_count; i++) {
        if (scheduler->sjf_bursts[i] < scheduler->sjf_bursts[min_index]) {
            min_index = i;
        }
    }

    int pid_elegido = scheduler->sjf_pids[min_index];
    // Eliminar el elemento elegido moviendo el último a su posición (O(1))
    scheduler->sjf_pids[min_index] = scheduler->sjf_pids[scheduler->sjf_count - 1];
    scheduler->sjf_bursts[min_index] = scheduler->sjf_bursts[scheduler->sjf_count - 1];
    scheduler->sjf_count--;

    return pid_elegido;
}
```

}

Explicación Narrativa: Para seleccionar el siguiente proceso, el algoritmo itera sobre un arreglo que contiene los procesos actualmente disponibles. Mediante un ciclo for, compara sus tiempos de ejecución. Una vez encontrado el índice con el menor valor (`min_index`), extrae el PID. Para evitar reacomodar todo el arreglo (lo cual costaría tiempo adicional), utiliza una técnica de intercambio en tiempo constante: reemplaza el proceso extraído con el proceso ubicado en la última posición del arreglo y decrementa el contador total.

Demostración Matemática: Sea k el número de procesos actualmente en espera dentro de la cola del planificador (`sjf_count`).

- Inicializar `min_index = 0` toma tiempo constante c_1 .
- El ciclo `for` arranca en $i = 1$ y termina en $k - 1$, ejecutándose $(k - 1)$ veces. La evaluación toma un tiempo c_2 .
- El reemplazo de posiciones de arreglos toma un tiempo c_3 .

$$T(k) = c_1 + \sum_{i=1}^{k-1} c_2 + c_3$$

$$T(k) = c_1 + c_2(k - 1) + c_3$$

$$T(k) = c_2 \cdot k + (c_1 + c_3 - c_2)$$

Despreciando los coeficientes y las constantes aditivas:

$$T(k) = O(k)$$

Programación Dinámica.

Análisis de matrices y dobles sumatorias anidadas.

Algoritmo: Programación Dinámica Bottom-Up (Optimización Knapsack)

La Programación Dinámica es un paradigma diseñado para resolver problemas de optimización complejos dividiéndolos en subproblemas superpuestos. En nuestro Sistema Operativo, aplicamos este algoritmo para resolver una variante del problema de la mochila (0/1 Knapsack Problem): encontrar el subconjunto de procesos que maximice la "prioridad" (o valor) total sin exceder la capacidad máxima de la RAM disponible, garantizando que el sistema ejecute los procesos más críticos primero.

Se eligió el enfoque *Bottom-Up* (Ascendente) mediante tabulación para evitar la recursión profunda y el recálculo redundante de subproblemas.

Código Esencial:

```
C
int dp_knapsack_processes(int ram_disponible, int pesos_memoria[], int prioridades[], int
n) {
    int i, w;
    // Asignación de memoria para la matriz DP (n+1) x (ram_disponible+1)
    int **dp = (int **)malloc((n + 1) * sizeof(int *));
    for (i = 0; i <= n; i++) {
        dp[i] = (int *)malloc((ram_disponible + 1) * sizeof(int));
    }

    // Llenado iterativo de la matriz
    for (i = 0; i <= n; i++) {
        for (w = 0; w <= ram_disponible; w++) {
            if (i == 0 || w == 0) {
                dp[i][w] = 0;
            } else if (pesos_memoria[i - 1] <= w) {
                dp[i][w] = max(prioridades[i - 1] + dp[i - 1][w - pesos_memoria[i - 1]], dp[i -
1][w]);
            } else {
                dp[i][w] = dp[i - 1][w];
            }
        }
    }

    int mejor_valor = dp[n][ram_disponible];
    // ... (Liberación de memoria omitida por brevedad) ...
    return mejor_valor;
}
```

Explicación Narrativa: El algoritmo construye una tabla bidimensional dp donde las filas representan los procesos evaluados (de 0 a n) y las columnas representan la capacidad de memoria simulada paso a paso (de 0 a $ram_disponible$ o W). A través de dos ciclos anidados, la matriz se llena desde la base ($i=0, w=0$). Para cada celda, el algoritmo toma una decisión local basada en soluciones previas ya almacenadas: evalúa si incluir el proceso actual genera un mayor valor (sumando su prioridad a la mejor combinación posible con el espacio restante) o si es preferible no incluirlo (heredando el valor de la fila superior). El resultado óptimo global siempre residirá en la última celda de la matriz $dp[n][W]$.

Demostración Matemática (Análisis por Doble Sumatoria): A diferencia de los algoritmos de una sola dimensión, la Programación Dinámica requiere evaluar el costo temporal en función de dos variables de entrada independientes: la cantidad de elementos n y la restricción de peso máxima W ($ram_disponible$).

Sea c_1 el costo constante de inicializar variables. Sea c_2 el costo constante de las comprobaciones condicionales `if/else`, operaciones aritméticas y la llamada a la función `max()` en el núcleo del ciclo.

El tiempo de ejecución $T(n, W)$ está dominado por los dos bucles `for` anidados:

$$T(n, W) = c_1 + \sum_{i=0}^n \left(\sum_{w=0}^W c_2 \right)$$

Resolvemos la sumatoria más interna (dependiente de w):

$$\sum_{w=0}^W c_2 = c_2 \cdot (W + 1)$$

Sustituimos en la sumatoria externa (dependiente de i):

$$T(n, W) = c_1 + \sum_{i=0}^n [c_2 \cdot (W + 1)]$$

Dado que el término interior es una constante con respecto a i , lo extraemos multiplicándolo por el número de iteraciones del bucle externo $(n + 1)$:

$$T(n, W) = c_1 + c_2 \cdot (W + 1) \cdot (n + 1)$$

$$T(n, W) = c_1 + c_2 \cdot (n \cdot W + n + W + 1)$$

Para determinar la complejidad asintótica (Big-O), nos enfocamos en el término de mayor crecimiento dominante $(n \cdot W)$ y descartamos los coeficientes constantes y términos lineales:

$$T(n, W) = \mathbf{O(n \cdot W)}$$

Análisis de Complejidad Espacial:

Dado que la tabulación requiere almacenar los resultados previos explícitamente para evitar recursiones redundantes, el algoritmo crea una matriz bidimensional mediante punteros a punteros (`int dp`).

La cantidad de elementos asignados dinámicamente es exactamente $(n + 1) \times (W + 1)$. Ignorando las constantes, esto nos da una complejidad de espacio linealmente proporcional al producto de ambas variables:

$$\text{Espacio} = \mathbf{O(n \cdot W)}$$

Planificadores y Estructuras Base.

Algoritmo: Planificador FIFO (Operaciones en Cola Lineal)

El planificador FIFO (First-In, First-Out) utiliza una Cola implementada mediante una Lista Ligada Simple. Su objetivo es mantener el orden de llegada de los procesos, garantizando que el primero en solicitar CPU sea el primero en ser ejecutado. La eficiencia de este planificador radica en su capacidad para insertar y extraer elementos sin necesidad de desplazar el resto de los datos.

Código Esencial:

C

```
void enqueue(Queue* queue, int item) {
    struct Node* new_node = malloc(sizeof(struct Node));
    new_node->data = item;
    new_node->next = NULL;

    if (queue->rear == NULL) {
        queue->front = queue->rear = new_node;
        return;
    }
    queue->rear->next = new_node;
    queue->rear = new_node;
}

int dequeue(Queue* queue) {
    struct Node* temp = queue->front;
    int data = temp->data;
    queue->front = queue->front->next;

    if (queue->front == NULL) queue->rear = NULL;
    free(temp);
    return data;
}
```

Explicación Narrativa: En la operación de inserción (enqueue), el algoritmo no recorre la lista para encontrar el final; en su lugar, utiliza el puntero auxiliar rear para enlazar directamente el nuevo nodo en la última posición. Similarmente, en la extracción (dequeue), el sistema no requiere desplazar los elementos restantes, simplemente avanza el puntero front al siguiente nodo y libera la memoria del nodo anterior. Al carecer de estructuras iterativas (ciclos for o while), el tiempo de ejecución es estrictamente constante, independientemente de si la cola tiene 10 o 10,000 procesos.

Demostración Matemática: Sea n el tamaño de la cola. Evaluamos el costo operativo de las instrucciones secuenciales:

Para enqueue(n):

- Asignación dinámica de memoria (malloc): constante c_1 .
- Asignación de variables (data, next): constante c_2 .
- Reasignación de punteros (rear): constante c_3 .

$$T_{\text{enqueue}}(n) = c_1 + c_2 + c_3$$

$$T_{\text{enqueue}}(n) = C \implies \mathbf{O(1)}$$

Para dequeue(n) :

- Lectura del puntero `front` y extracción de datos: constante c_4 .
- Actualización del puntero `front` : constante c_5 .
- Liberación de memoria (`free`): constante c_6 .

$$T_{\text{dequeue}}(n) = c_4 + c_5 + c_6$$

$$T_{\text{dequeue}}(n) = K \implies \mathbf{O(1)}$$

Algoritmo: Planificador Round Robin (Operaciones en Cola Circular)

El planificador Round Robin es el motor principal de nuestro Sistema Operativo. Funciona asignando a cada proceso un intervalo de tiempo fijo (Quantum). Cuando el Quantum expira, el proceso es interrumpido y reinsertado al final de la fila. Para implementar esto eficientemente sin agotar la memoria con asignaciones dinámicas constantes, se utilizó una Cola Circular basada en arreglos estáticos y aritmética modular.

Código Esencial:

C

```
void cq_enqueue(CircularQueue* cq, int item) {
    if (cq_is_full(cq)) return;
    // Aritmética modular para el comportamiento circular
    cq->rear = (cq->rear + 1) % cq->capacity;
    cq->data[cq->rear] = item;
    cq->size++;
}

int cq_dequeue(CircularQueue* cq) {
    if (cq_is_empty(cq)) return -1;
    int item = cq->data[cq->front];
    // Aritmética modular para avanzar el frente
    cq->front = (cq->front + 1) % cq->capacity;
    cq->size--;
    return item;
}
```

Explicación Narrativa: La principal ventaja de la Cola Circular radica en el uso de la operación matemática de módulo (%). Cuando el puntero rear llega al límite del arreglo físico (capacity), la operación $(rear + 1) \% capacity$ reinicia automáticamente el puntero a la posición 0, reutilizando los espacios que ya fueron liberados por dequeue al frente del arreglo. Esto evita el costoso proceso $O(n)$ de desplazar todos los elementos a la izquierda.

Demostración Matemática: Sea n la cantidad de elementos actuales en la cola circular y N su capacidad máxima (capacity).

Evaluamos $cq_enqueue(n)$ y $cq_dequeue(n)$:

- Evaluación de condiciones límite (if full/empty): constante $c1$.
- Cálculo aritmético de la nueva posición $(x + 1) \% N$: la suma y el módulo en el procesador ALU toman un tiempo fijo $c2$.
- Lectura/Escritura en el índice del arreglo ($data[index]$): el acceso directo a memoria toma tiempo constante $c3$.
- Incremento/Decremento del tamaño (size): constante $c4$.

La función de costo temporal es independiente de n :

$$T(n) = c_1 + c_2 + c_3 + c_4$$

$$T(n) = C$$

Aplicando el límite asintótico:

$$T(n) = \mathbf{O(1)}$$

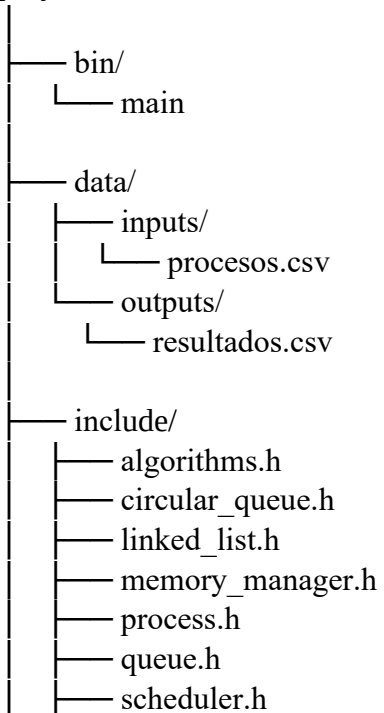
Conclusión: La gestión de procesos mediante Cola Circular asegura que cada cambio de contexto (interrupción por Quantum) se ejecute en tiempo estrictamente constante $O(1)$. La complejidad espacial está predefinida por la capacidad máxima del arreglo, siendo $O(N)$ globalmente, pero $O(1)$ por cada operación de encolado o extracción.

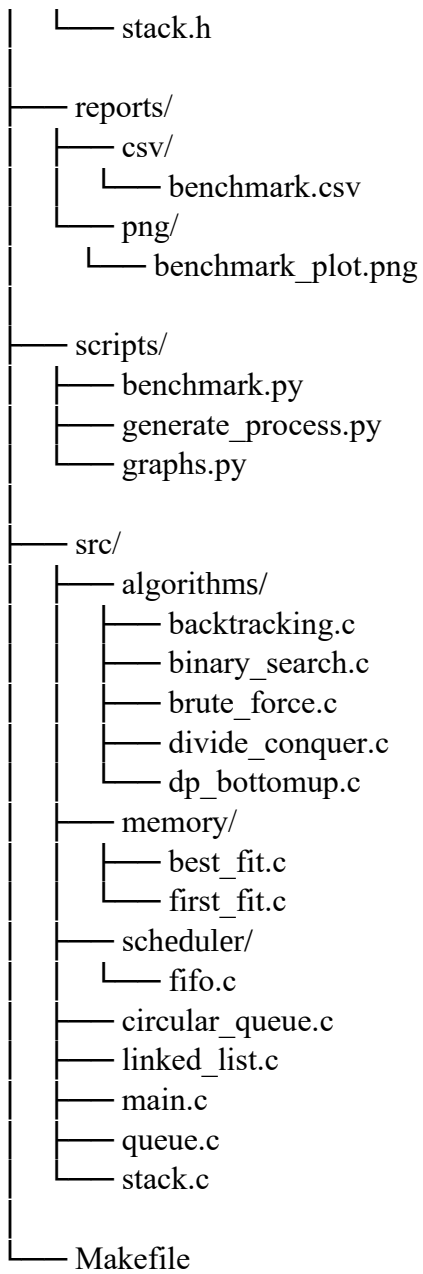
SIMPLIFICACION Y RESUMEN DEL REPORTE TECNICO

Simplificacion de Reporte Tecnico

Orden de los programas

proyecto-final/





Orden con que hace cada parte.

proyecto-final/

├── **bin/**
│ └── **main** (El programa final de unos y ceros listo para que la computadora lo corra).

├── **data/**
│ ├── **inputs/**
│ │ └── **procesos.csv** (El archivo de texto donde escribes a mano los procesos que quieres probar).

│ └── **outputs/**
│ └── **resultados.csv** (Aquí el C escupe su reporte después de correr la simulación).

├── **include/** (Los "contratos" o índices que conectan las piezas de código entre sí).

│ ├── **algorithms.h**
│ ├── **circular_queue.h**
│ ├── **linked_list.h**
│ ├── **memory_manager.h**
│ ├── **process.h**
│ ├── **queue.h**
│ ├── **scheduler.h**
│ └── **stack.h**

├── **reports/**
│ ├── **csv/**
│ │ └── **benchmark.csv** (El archivo que crea Python midiendo cuántos milisegundos tardó tu código).

│ └── **png/**
│ └── **benchmark_plot.png** (La imagen de la gráfica azul generada por Python).

├── **scripts/**
│ ├── **benchmark.py** (Script que pone a sudar al programa en C mandándole miles de procesos).

│ ├── **generate_process.py** (Script que inventa procesos aleatorios rápido para no escribirlos a mano).

│ └── **graphs.py** (Script que dibuja la gráfica visual leyendo el benchmark.csv).

└── **src/**
 ├── **algorithms/**
 │ └── **backtracking.c** (Guarda intentos de memoria en una pila y retrocede si el intento falla).

```

|  |  | — binary_search.c  (Busca un proceso rapidísimo partiendo la lista a la mitad
|  |  | — brute_force.c    (Busca el peor hueco de memoria revisando todo de principio
|  |  | — divide_conquer.c (Ordena la memoria: avienta los ocupados a un lado y los
|  |  | — dp_bottomup.c     (Calcula inteligentemente qué procesos son más valiosos
|  |  | — memory/
|  |  | — best_fit.c       (Busca el hueco de RAM más apretado para no dejar casi
|  |  | — first_fit.c      (Toma el primer hueco de RAM que le sirva y no pierde tiempo
|  |  | — scheduler/
|  |  | — fifo.c           (Aquí viven las reglas para ordenar procesos por FIFO, Round
|  |  | — circular_queue.c (Una fila en forma de círculo infinito para que el Round
|  |  | — linked_list.c    (Una lista de "vagones de tren" para guardar los procesos que
|  |  | — main.c           (EL CEREBRO: Prende el reloj, junta todas las piezas y corre el
|  |  | — queue.c          (Una fila normal donde el primero que llega es el primero en
|  |  | — stack.c          (Una pila tipo platos para ayudar al algoritmo de Backtracking).
|  |  | — Makefile         (El manual de instrucciones para que la PC compile todo con solo
|  |  |                      escribir "make").

```

Resumen de Complejidad Algoritmica

- **Búsqueda Binaria** - Divide y Vencerás - Teorema Maestro - **$O(\log n)$**
- **Compactación de Memoria** - Divide y Vencerás - Sumatoria - **$O(n)$**
- **Worst Fit (Peor Ajuste)** - Fuerza Bruta - Sumatoria - **$O(n)$**
- **Asignación de Memoria con Pila** - Backtracking - Sumatoria - **$O(n)$**
- **First Fit (Primer Ajuste)** - Greedy (Avaro) - Sumatoria (Peor Caso) - **$O(n)$**
- **Best Fit (Mejor Ajuste)** - Greedy (Avaro) - Sumatoria - **$O(n)$**
- **Shortest Job First (SJF)** - Greedy (Avaro) - Sumatoria - **$O(n)$**
- **Mochila (Knapsack)** - Programación Dinámica (Bottom-Up) - Doble Sumatoria - **$O(n * W)$**
- **Planificador FIFO** - Estructura Base (Cola Lineal) - Conteo de Instrucciones - **$O(1)$**
- **Planificador Round Robin** - Estructura Base (Cola Circular) - Conteo de Instrucciones - **$O(1)$**

Conclusion general del sistema

La complejidad asintótica general del sistema está dominada por su algoritmo más pesado, la Programación Dinámica, dándole un límite superior teórico de **$O(n \times W)$** . Sin embargo, la arquitectura está diseñada para que este esfuerzo se realice solo durante la optimización inicial. Durante el ciclo de vida continuo del procesador, gracias a las Colas Circulares $O(1)$ y la Búsqueda Binaria $O(\log n)$, el cuello de botella en tiempo real se reduce exclusivamente a la búsqueda de memoria **$O(m)$** , logrando un simulador altamente escalable y eficiente.

Benchmark

```
import subprocess
import time
import csv

def run_benchmark():
    # Diferentes cargas de procesos para medir escalabilidad
    tamanos = [100, 500, 1000, 5000]
    resultados = []

    print("Iniciando Benchmark...")
    for n in tamanos:
        # Medimos el tiempo que tarda el ejecutable de C
        inicio = time.time()
        # Llamamos al ejecutable (asegúrate de que esté en bin/main)
        subprocess.run(["./bin/main.exe", str(n)], stdout=subprocess.DEVNULL)
        fin = time.time()

        tiempo_ms = (fin - inicio) * 1000
        resultados.append([n, tiempo_ms])
        print(f"Carga de {n} procesos completada en {tiempo_ms:.2f} ms")

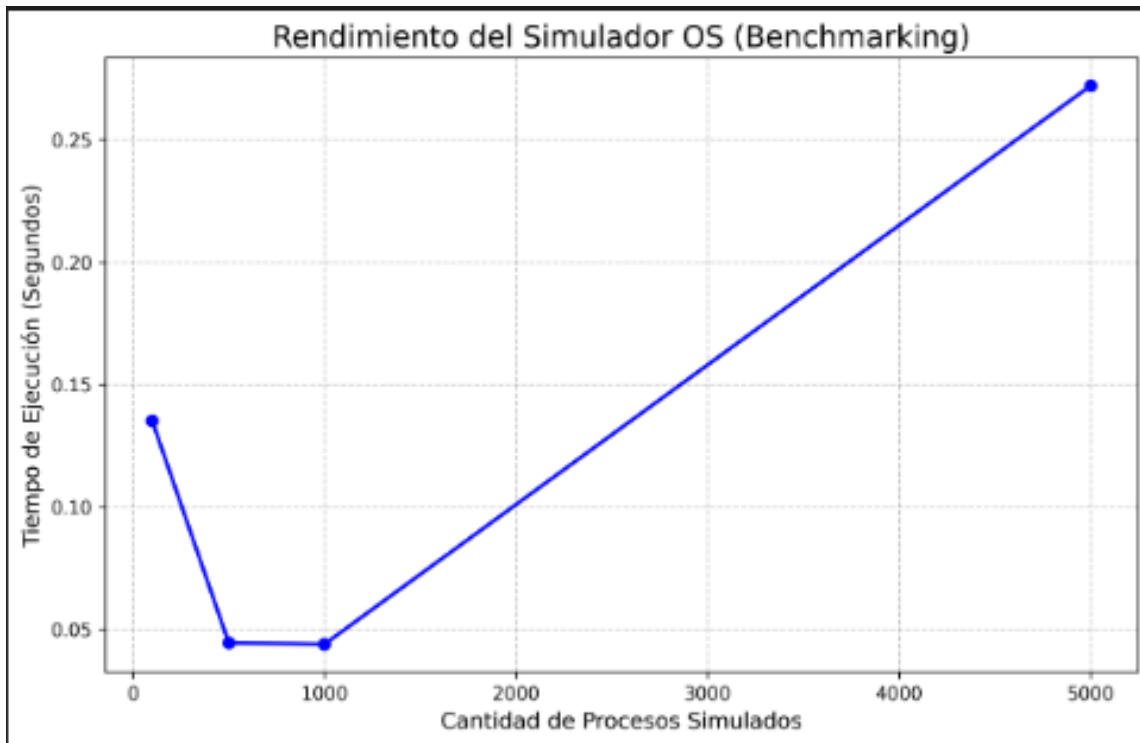
    # Guardamos los resultados
    with open("reports/csv/benchmark.csv", "w", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(["Procesos", "Tiempo_ms"])
        writer.writerows(resultados)
    print("Benchmark guardado en reports/csv/benchmark.csv")

if __name__ == "__main__":
    run_benchmark()
```

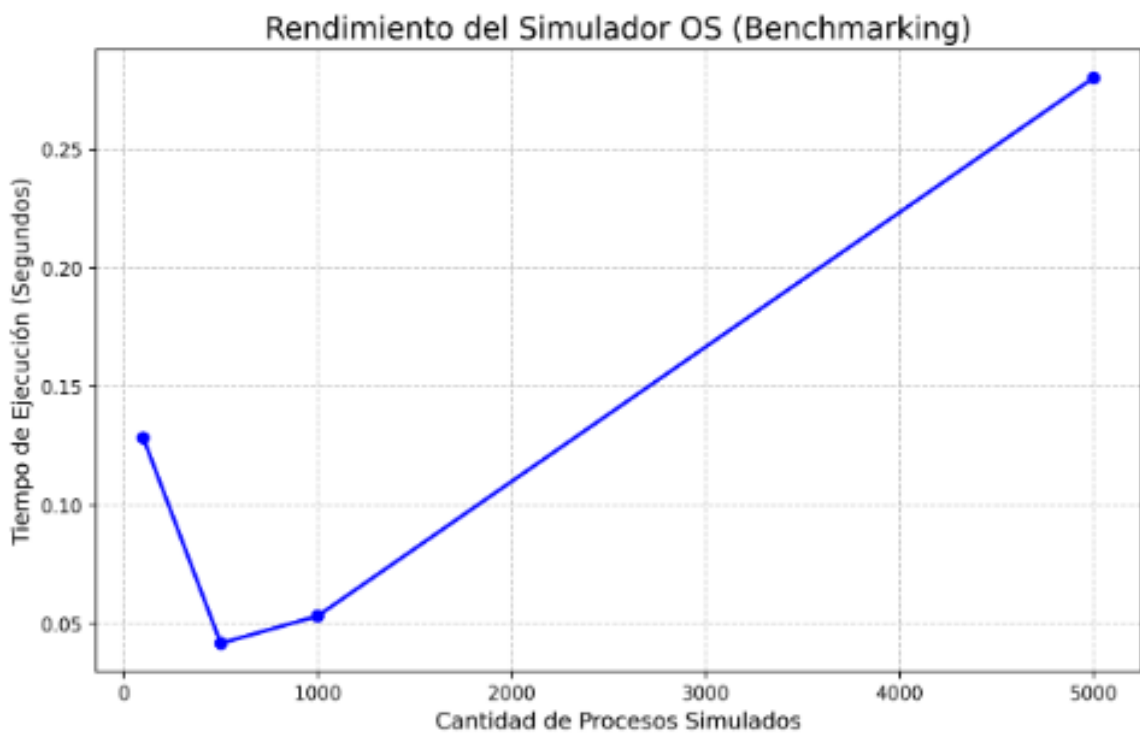
Graficas

Bajo las mismas condiciones de recursos, siendo una RAM de 1024 y Quantum de 2
Simulando 100, 500, 1000 y 5000 procesos

Con el first fit



Con el best fit



El first fit corresponde mejor a mayor cantidad de datos

Conclusion

Para esta prueba sencilla, se observan la diferencia entre los algoritmos en concreto, su velocidad a la hora de manejar datos tan grandes, aun que si son mas pequenos estos son básicamente iguales con la unica diferencia que el First Fit tiene una velocidad de “arranque” , como si fuera una computadora recién encendida, por ello es un poco menos veloz con datos pequenos

- **First Fit** es ligeramente más rápido (0.272s) porque simplemente recorre la memoria y en el momento que encuentra un hueco donde el proceso cabe, lo mete ahí y deja de buscar. Pierde menos tiempo procesando.
- **Best Fit** es un poco más lento (0.280s) porque, aunque encuentre un hueco rápido, tiene la obligación matemática de seguir buscando por **toda** la memoria para asegurarse de encontrar el hueco "más perfecto" (el que deje menos espacio desperdiciado). Esa búsqueda extra le cuesta tiempo de CPU.